```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
print("Upload Advertising.csv")
uploaded = files.upload()

filename = next(iter(uploaded))
df = pd.read_csv(filename)

# Clean column names
df.columns = df.columns.str.strip()

# Remove any accidental index column
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]

print("\nFIRST 5 RECORDS")
display(df.head())

print("\nLAST 5 RECORDS")
display(df.tail())

print("\nDATASET SHAPE:", df.shape)

print("\nDATA TYPES:")
print(df.dtypes)

print("\nMISSING VALUES:")
print(df.isnull().sum())

print("\nDUPLICATE RECORDS:", df.duplicated().sum())

print("\nSUMMARY STATISTICS:")
display(df.describe())


# Remove duplicate records and missing values
df = df.drop_duplicates().dropna().copy()

# Select features and target
features = ["TV", "Radio", "Newspaper"]
target = "Sales"

X = df[features]
y = df[target]

print("\nINPUT FEATURES:")
display(X.head())

print("\nTARGET VARIABLE:")
display(y.head())
```

Upload Advertising.csv

Choose Files archive (1).zip

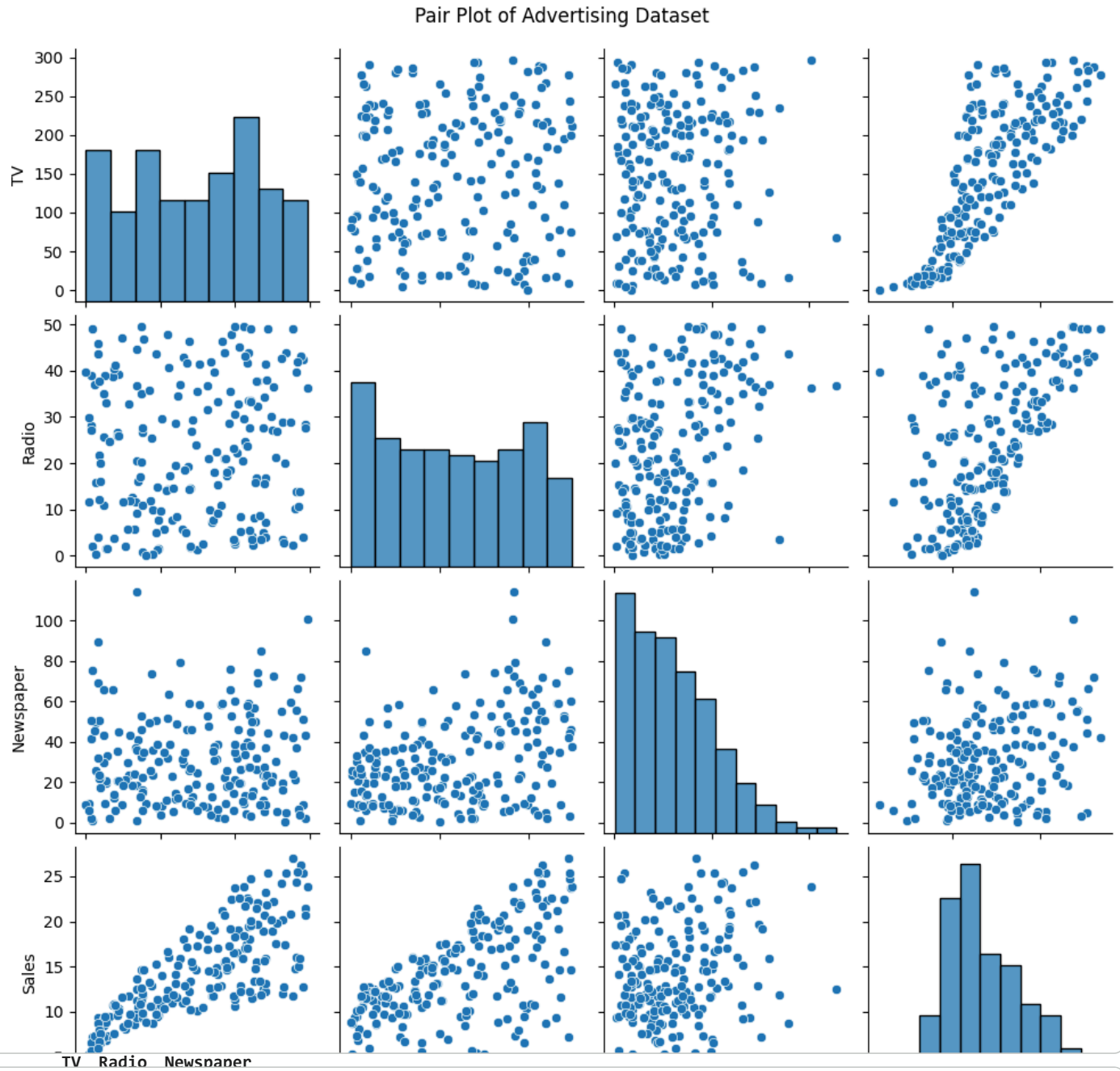
archive (1).zip(application/x-zip-compressed) - 2316 bytes, last modified: 10/9/2026 - 100% done

Saving archive (1).zip to archive (1).zip

FIRST 5 RECORDS

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5

```
sns.pairplot(df[features + [target]])
plt.suptitle("Pair Plot of Advertising Dataset", y=1.02)
plt.show()
```



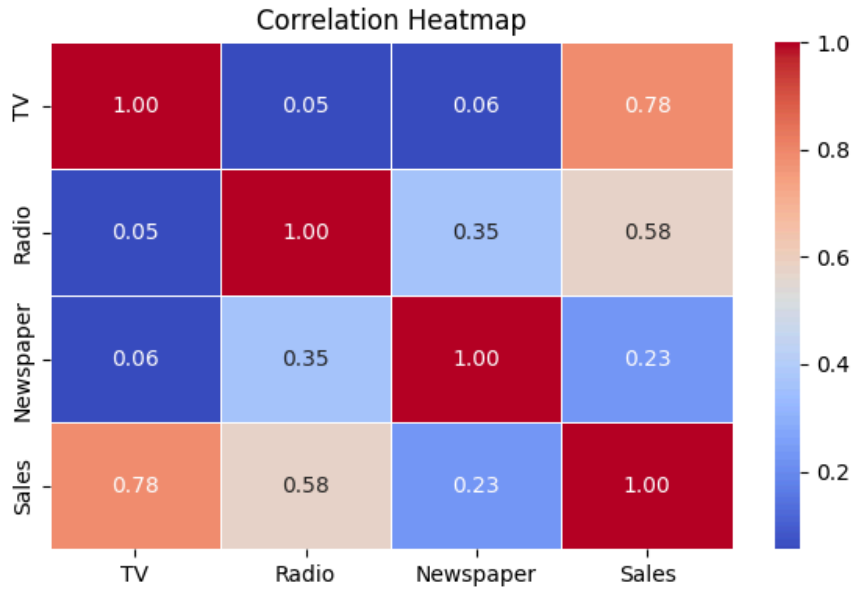

```
correlation = df[
    ["TV", "Radio", "Newspaper", "Sales"]
].corr()

plt.figure(figsize=(6, 4))

sns.heatmap(
    correlation,
```

```
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```




```
sales_correlation = correlation["Sales"].drop("Sales")

strongest_feature = sales_correlation.abs().idxmax()

print("\nCORRELATION OF FEATURES WITH SALES")
print(sales_correlation)

print("\nFEATURE MOST STRONGLY CORRELATED WITH SALES:")
print(strongest_feature)

print(
    "Correlation value:",
    round(sales_correlation[strongest_feature], 4)
)
```

```
CORRELATION OF FEATURES WITH SALES
TV           0.782224
Radio        0.576223
Newspaper    0.228299
Name: Sales, dtype: float64

FEATURE MOST STRONGLY CORRELATED WITH SALES:
TV
Correlation value: 0.7822
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTRAINING DATA SHAPE:", X_train.shape)
print("TESTING DATA SHAPE:", X_test.shape)
print("Training target shape:", y_train.shape)
print("Testing target shape:", y_test.shape)
```

```
TRAINING DATA SHAPE: (160, 3)
TESTING DATA SHAPE: (40, 3)
Training target shape: (160,)
Testing target shape: (40,)
```

```
linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

linear_pred = linear_model.predict(X_test)

print("\nLINEAR REGRESSION")
print("Model coefficients:", linear_model.coef_)
print("Intercept:", linear_model.intercept_)

linear_results = pd.DataFrame({
    "Actual Sales": y_test.to_numpy(),
    "Predicted Sales": linear_pred
})

display(linear_results.head(10))
```

```
LINEAR REGRESSION
Model coefficients: [0.04472952 0.18919505 0.00276111]
Intercept: 2.979067338122629
```

	Actual Sales	Predicted Sales
0	16.9	16.408024
1	22.4	20.889882
2	21.4	21.553843
3	7.3	10.608503
4	24.7	22.112373
5	12.6	13.105592
6	22.3	21.057192
7	8.4	7.461010
8	11.5	13.606346
9	14.9	15.155070

```
polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    LinearRegression()
)

polynomial_model.fit(X_train, y_train)

polynomial_pred = polynomial_model.predict(X_test)
```

```
print("\nPOLYNOMIAL REGRESSION (DEGREE 2)")

polynomial_results = pd.DataFrame({
    "Actual Sales": y_test.to_numpy(),
    "Predicted Sales": polynomial_pred
})

display(polynomial_results.head(10))
```

```
POLYNOMIAL REGRESSION (DEGREE 2)
```

	Actual Sales	Predicted Sales
0	16.9	17.254436
1	22.4	22.719332
2	21.4	20.427994
3	7.3	7.542709
4	24.7	24.371030
5	12.6	12.559272
6	22.3	22.807850
7	8.4	8.343714
8	11.5	12.058602
9	14.9	15.690246

```
svr_model = make_pipeline(
    StandardScaler(),
    SVR(kernel="rbf")
)

svr_model.fit(X_train, y_train)

svr_pred = svr_model.predict(X_test)

print("\nSUPPORT VECTOR REGRESSION (RBF KERNEL)")

svr_results = pd.DataFrame({
    "Actual Sales": y_test.to_numpy(),
    "Predicted Sales": svr_pred
})

display(svr_results.head(10))
```

```
SUPPORT VECTOR REGRESSION (RBF KERNEL)
```

	Actual Sales	Predicted Sales
0	16.9	17.187960
1	22.4	21.604973
2	21.4	19.580206
3	7.3	9.766378
4	24.7	19.954920
5	12.6	12.496915
6	22.3	21.906620
7	8.4	8.540921
8	11.5	12.053972
9	14.9	15.486639

```
Start coding or generate with AI.
```